A Hybrid AsyncIO and Multiprocessing Approach for Bloom Filter Acceleration in Python

Alessio Poggesi
alessio.poggesi@edu.unifi.it

Abstract

This work presents a comparison between a sequential Bloom Filter implementation and a hybrid parallel version in Python that combines asyncio for asynchronous file I/O and multiprocessing for CPU-bound computation. The workload consists of constructing the Bloom Filter from large-scale datasets and performing membership queries, while varying input size, dataset fragmentation, number of worker processes, and chunking strategies.

The parallel design separates I/O-bound and CPU-bound tasks: file reading is handled asynchronously to avoid blocking the event loop, while hash computation and bit array updates are parallelized across multiple processes using a map–reduce approach. Experimental results show that, for sufficiently large datasets, the hybrid implementation achieves a stable speedup of approximately 4× using 8 worker processes on datasets of 10 and 20 million elements, demonstrating effective strong scaling and controlled parallel overhead. These findings highlight how a carefully balanced hybrid approach can overcome Python's Global Interpreter Lock limitations in data-intensive applications. The evaluation considers both end-to-end timing and a complementary compute-only measurement, allowing a clearer separation between file I/O overhead and the actual cost of computation and parallel coordination.

Future Distribution Permission

The author of this report gives permission for this document to be distributed to Unifi-affiliated students taking future courses.

1. Introduction

Bloom Filters are probabilistic data structures used to efficiently test set membership while maintaining a compact memory footprint. They are widely adopted in large-scale systems such as databases, caching mechanisms, and distributed applications, where fast membership queries are required and a limited probability of false positives is acceptable. A Bloom Filter relies on a fixed-size bit array and multiple hash functions to insert and query elements.

When applied to large datasets, Bloom Filters can become computationally demanding. Both insertion and query operations require the repeated evaluation of several hash functions per element, resulting in a predominantly CPU-bound workload. Additionally, reading large amounts of data from disk introduces a non-negligible I/O overhead.

In Python, these challenges are exacerbated by the Global Interpreter Lock (GIL), which limits parallel execution of CPU-bound threads. As a result, sequential implementations scale poorly to millions of elements, while naive parallelization may incur significant overhead.

This work investigates whether a hybrid parallel approach based on `asyncio` and `multiprocessing` can improve Bloom Filter performance compared to a sequential baseline by separating I/O-bound and CPU-bound tasks and identifying the configurations in which parallelization provides effective performance gains. In addition to the traditional end-to-end execution time, the experimental evaluation also considers a complementary compute-only timing perspective, aimed at isolating the algorithmic and parallelization costs from file I/O and parsing overhead.

2. Algorithm Description

2.1. Sequential Bloom Filter Algorithm

A Bloom Filter is a probabilistic data structure that represents a set using a fixed-size bit array of

length m and a set of k independent hash functions. Initially, all bits in the array are set to zero. Each hash function maps an input element to an index in the range $[0, m-1]$.

During the *insertion phase*, each element from the input dataset is processed sequentially. For a given element, the k hash functions are evaluated, producing k indices. The bits at these positions in the bit array are then set to one. This operation does not require checking previous values, making insertion simple and efficient.

During the *query phase*, the same k hash functions are applied to the queried element. If all corresponding bits in the bit array are set to one, the element is reported as *possibly present*; if at least one bit is zero, the element is *definitely not present*. Due to hash collisions, Bloom Filters may produce false positives, but never false negatives.

In the sequential implementation, both insertion and querying iterate linearly over the input elements. The overall computational cost is dominated by the repeated evaluation of hash functions, resulting in a time complexity of $O(n \cdot k)$, where n is the number of processed elements. Memory usage is fixed and depends only on the chosen values of m and k.

2.2. Identification of Parallelizable Sections

The Bloom Filter algorithm exhibits a high degree of inherent parallelism. Each element is processed independently, both during insertion and querying, as hash computations for one element do not depend on the results of others. This makes the algorithm *embarrassingly parallel* at the level of input elements.

However, direct parallel updates to a shared bit array introduce potential data races and synchronization overhead, as multiple workers may attempt to set the same bit concurrently. While such races are logically harmless (setting a bit to one is an idempotent operation), their management in Python would require locks or atomic operations, significantly reducing performance.

To avoid fine-grained synchronization, the parallelizable sections are identified as the hash computation and index generation for each element, while the update of the global bit array is deferred. In a parallel design, each worker can process a subset of the input elements and construct a *local Bloom Filter* by setting bits in a private bit array. These partial results can then be merged safely in a reduction step using bitwise OR operations.

Similarly, during the query phase, membership tests for different elements can be performed independently on separate workers, as they only require read-only access to the Bloom Filter. This separation of independent computation and controlled aggregation forms the basis for the parallel implementation described in the following section.

3. Parallel Implementation

3.1. Parallelization Strategy

The parallel implementation adopts a hybrid strategy that combines `asyncio` and `multiprocessing` to effectively handle both I/O-bound and CPU-bound components of the Bloom Filter workload. The main idea is to separate file reading from hash computation and bit array manipulation, allowing these phases to be handled through distinct mechanisms within the same execution flow.

Asynchronous programming through `asyncio` is used to manage file reading and task coordination without blocking the main execution flow. Computationally intensive operations, such as hash evaluation and bit array updates, are instead parallelized using multiple processes through a `ProcessPoolExecutor`, thus bypassing the limitations imposed by Python's Global Interpreter Lock (GIL). The overall approach follows a *map–reduce* pattern, where independent portions of the input are processed in parallel and later aggregated.

3.2. Use of Multiprocessing in the Algorithm

Multiprocessing is employed in the parts of the algorithm where computational load is dominant, particularly during the Bloom Filter construction phase and the membership query phase. In both

cases, input elements are divided into chunks, each assigned to a separate worker process.

During the construction phase, each worker processes its assigned subset of elements by computing the corresponding hash values and setting the associated bits in a local bit array. This design avoids concurrent access to a shared data structure, eliminating the need for locks or fine-grained synchronization. Once parallel processing is completed, the local bit arrays produced by the workers are returned to the main process in serialized form and merged into a global Bloom Filter using bitwise OR operations, which are efficient and deterministic.

In the query phase, the global Bloom Filter is accessed in read-only mode. Membership tests for different elements can therefore be executed in parallel by separate workers without introducing synchronization issues. Chunking is again used to reduce scheduling overhead and to improve load balancing across processes.

The integration with `asyncio` enables efficient coordination of these parallel activities. The event loop manages asynchronous I/O operations and the submission of CPU-bound tasks to external executors, ensuring that the main execution flow remains responsive. Synchronization between processes is handled implicitly by waiting for all tasks to complete before proceeding to aggregation steps, keeping the implementation simple and free of race conditions.

This approach allows effective exploitation of process-level parallelism while limiting communication and synchronization overhead. At the same time, it makes it possible to distinguish between global end-to-end costs and compute-only costs associated with chunking, task submission, worker execution, and result aggregation, which are all relevant to the experimental evaluation.

4. Experimental Setup

4.1. Hardware

All experiments were conducted on a single workstation running a 64-bit operating system. The system is equipped with an **11th Gen Intel(R) Core(TM) i7-11800H** processor with a base frequency of **2.30 GHz**, featuring **8 physical cores** and **16 hardware threads**. The machine is equipped with **16 GB of RAM**, which is sufficient to store both the Bloom Filter bit arrays and the large input datasets entirely in main memory, avoiding disk swapping.

4.2. Software Environment

The entire project was developed in **Python**. Parallelism was implemented exclusively using standard Python libraries:

- `asyncio`, used for asynchronous I/O management and task coordination;
- `multiprocessing`, through `ProcessPoolExecutor`, used for parallel execution of CPU-bound tasks.

For hash computation within the Bloom Filter, the external library **MurmurHash3**, accessed via the Python package `mmh3`, was used. This library was chosen for its high performance and good hash value distribution. The Bloom Filter bit array is implemented using a dedicated bit array library that provides compact memory representation and efficient bitwise operations.

All experiments were executed using the same Python interpreter and library versions. Garbage collection was explicitly triggered between runs to reduce interference caused by automatic memory management.

4.3. Datasets

All datasets used in the experiments are **fully synthetic** and are generated using the script `generate_data.py`, located in the root directory of the project. The script is designed to produce controlled and reproducible workloads, allowing systematic variation of dataset size and input fragmentation.

For each configuration, two distinct datasets are generated:

- a **training dataset**, used to build the Bloom Filter;

- a **test dataset**, used during the membership query phase.

Each dataset element is a randomly generated alphanumeric string with a length uniformly distributed between 10 and 20 characters. Characters are sampled from a fixed alphabet consisting of lowercase letters and digits. Training and test datasets have the **same size** but contain **independent data**, with no intentional overlap between the two sets. As a result, most membership queries are expected to be negative, allowing the evaluation of Bloom Filter behavior in the presence of false positives.

Generated datasets are stored in the directory `bench_data/`, which is organized hierarchically according to dataset size and fragmentation level. The directory structure is as follows:

```
bench_data/
  size_<N>/
      split_<S>/
          train/
              train_00.txt
              train_01.txt
              ...
          test/
               test_00.txt
               test_01.txt
               ...
```

Here, `size_<N>` denotes the total dataset size (2M, 5M, 10M, or 20M elements), while `split_<S>` represents the number of files used to store the dataset. Although the dataset generator supports multiple fragmentation levels, the final benchmark reported in this work evaluates the configurations $S \in \{1, 4, 16\}$. Dataset fragmentation does not alter the dataset content, but only its on-disk organization, enabling the study of I/O behavior and asynchronous file access.

4.4. Tested Parameters

The experimental evaluation systematically explores the following parameters:

- **Dataset size**: 2M, 5M, 10M, and 20M elements;
- **Number of worker processes**: 1, 4, 8, and 16;
- **Chunk size**: fixed values (20,000 and 50,000 elements) and an automatic mode;
- **Dataset fragmentation**: number of input files, with tested values $S \in \{1, 4, 16\}$.

Bloom Filter parameters, specifically the bit array size and the number of hash functions, are computed automatically as a function of dataset size and a target false positive rate of 1%.

4.5. Measurement Methodology

Performance measurements are carried out using the script `main.py`, which implements a complete and reproducible benchmark. For each configuration, both the **sequential implementation** (`sequential_bloom_filter.py`) and the **hybrid parallel implementation** (`hybrid_bloom_filter.py`) are executed using identical datasets and parameters.

Two complementary timing perspectives are collected.

End-to-end timing. Execution time is measured using the high-resolution wall-clock timer `time.perf_counter`. This measurement includes the full workflow of each implementation, namely file access, parsing, Bloom Filter construction, membership queries, and all coordination overheads introduced by the hybrid approach.

Compute-only timing. In addition to the end-to-end measurement, the implementation records an internal compute-only time for both the build and verify phases. This second measure excludes file reading and parsing when possible, while retaining the algorithmic and parallelization costs. In the sequential version, compute-only timing isolates the cost of hash computation and bit-array operations during insertion and querying. In the hybrid version, it excludes asynchronous file reading and parsing, but

still includes chunking, task submission to the `ProcessPoolExecutor`, worker execution, synchronization through `gather`, and final aggregation of partial results.

For each configuration, an initial **warm-up run** is performed and excluded from the measurements, followed by **three measured runs**. Reported values correspond to the average execution time across the measured runs.

Accordingly, two speedup definitions are considered in the analysis:

- an **end-to-end speedup**, computed from the total execution times of the sequential and hybrid implementations;
- a **compute-only speedup**, computed from the corresponding compute-only times.

Final results are stored in CSV files and are used for the performance analysis presented in the following section.

5. Results and Analysis

5.1. Correctness Validation

Correctness of the parallel implementation was verified by comparing its behavior against the sequential Bloom Filter baseline. For each experimental configuration, the hybrid implementation was executed using the same training and test datasets as the sequential version.

The validation focused on ensuring that:

- the Bloom Filter construction completed without errors;
- membership queries produced consistent results with the sequential implementation;
- no false negatives were observed, as guaranteed by the Bloom Filter properties.

Since training and test datasets were generated independently, most membership queries are expected to return negative results. Any positive response from the Bloom Filter was therefore interpreted as a false positive, which is acceptable by design. Identical query behavior between sequential and hybrid implementations confirmed the correctness of the parallelization strategy.

5.2. Performance Results

Performance results are reported in terms of execution time and speedup relative to the sequential baseline. In the updated benchmark, two complementary perspectives are considered:

- **end-to-end performance**, which includes the full workflow of each implementation;
- **compute-only performance**, which isolates the algorithmic and parallelization costs while excluding file reading and parsing when possible.

For end-to-end measurements, speedup is defined as

$$\text{Speedup} = \frac{T_{\text{sequential}}}{T_{\text{parallel}}}$$

where $T_{\text{sequential}}$ and T_{parallel} denote the total execution time (build + query) of the sequential and hybrid implementations, respectively.

Across all dataset sizes, the hybrid implementation outperforms the sequential baseline for the large majority of tested configurations. The best end-to-end results are obtained with automatic chunk sizing and 8 worker processes. Under this setting, speedup grows from approximately $2.79\times$ on the 2M dataset to about $4.22\times$ on the 20M dataset. This confirms that the proposed hybrid strategy becomes increasingly effective as workload size grows.

The compute-only analysis shows an even stronger scaling behavior. With the same representative configuration (single split, automatic chunking, 8 workers), compute-only speedup increases from about $3.55\times$ on 2M data to approximately $5.75\times$ on 20M data. This difference indicates that the core computational part of the algorithm scales better than the full end-to-end workflow, and that a non-negligible fraction of the total cost is due to I/O and coordination overheads rather than to the Bloom Filter computation itself.

5.3. Speedup Curves

Speedup curves were obtained by varying the number of worker processes while keeping the

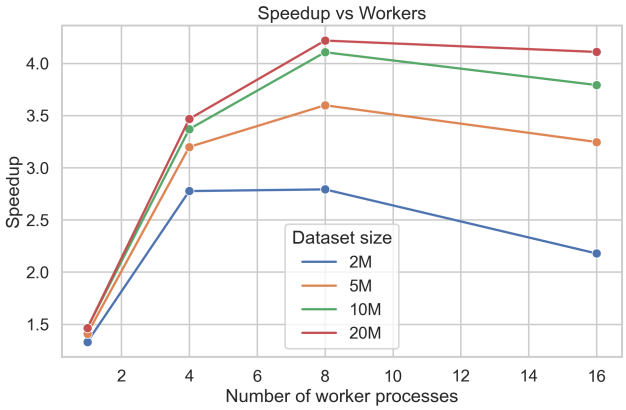

Figure 1. Speedup as a function of the number of worker processes for different dataset sizes, using a single input split and automatic chunk sizing.

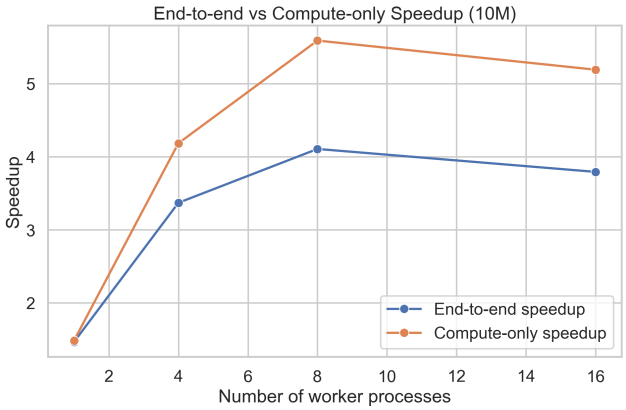

Figure 2. Comparison between end-to-end speedup and compute-only speedup for the 10M dataset, using a single input split and automatic chunk sizing.

dataset size fixed. Figure 1 reports the end-to-end speedup achieved by the hybrid implementation for different input sizes, using automatic chunk sizing and a single input split in order to isolate the effect of parallelism.

For all dataset sizes, end-to-end speedup increases as the number of workers grows up to a clear saturation point. For the smallest dataset (2M elements), the maximum speedup remains below $3\times$, indicating that multiprocessing overhead still represents a significant fraction of the total execution time. As the dataset size increases, the effectiveness of parallelization improves substantially. For the 10M and 20M datasets, the hybrid implementation reaches about $4.11\times$ and $4.22\times$ speedup, respectively, with 8 worker processes.

Increasing the number of workers beyond 8 does not lead to further performance improvements and typically results in a slight degradation. This behavior is consistent across dataset sizes and suggests that, on the tested hardware, 8 worker processes provide the best balance between useful parallelism and process-management overhead.

To better interpret these results, Figure 2 compares end-to-end and compute-only speedup for the 10M dataset, again using a single input split and automatic chunk sizing. The two curves follow the same overall trend, with both improving up to 8 workers and then saturating. However, the compute-only curve is systematically higher, reaching approximately $5.59\times$ compared to about $4.11\times$ for the end-to-end case.

This gap is expected and confirms that the computational core of the hybrid implementation scales more efficiently than the full workflow. In other words, once file reading and parsing overhead are excluded, the benefits of multiprocessing become even more evident. At the application level, part of the achievable speedup is therefore limited not by the Bloom Filter computation itself, but by I/O and coordination costs.

5.4. Scalability Analysis

The results provide clear evidence of effective **strong scaling**. As the number of worker processes increases, execution time decreases substantially up to 8 workers for all dataset sizes considered. This behavior is especially evident for the larger workloads, where the computational part dominates the fixed overheads introduced by multiprocessing.

The most stable and effective scaling behavior is observed on the 10M and 20M datasets. In the end-to-end analysis, both reach speedups slightly above $4\times$ at 8 workers. The compute-only analysis further strengthens this conclusion, showing that the same datasets exceed $5.5\times$ speedup at the same operating point. This indicates that the algorithmic core of the hybrid solution scales well and that the main scalability limits arise outside

the pure computation.

Scalability saturates around 8 worker processes, despite the availability of 16 hardware threads on the target machine. Beyond this point, additional processes do not translate into proportional reductions in execution time and often cause a mild performance regression. This effect is consistent with increased context switching, contention for shared hardware resources, and inter-process coordination overhead.

Overall, the scalability analysis confirms that the proposed hybrid approach is well balanced and particularly effective for large, CPU-bound Bloom Filter workloads, with 8 worker processes representing the best overall configuration on the tested platform.

5.5. Effect of Different Parameters

This section analyzes the impact of different parameters on the performance of the hybrid Bloom Filter implementation. In particular, we focus on how execution time scales with the size of the input dataset and on the effect of chunk size selection. The influence of worker count has already been discussed through the speedup curves presented in the previous subsection.

Execution Time Scaling with Input Size. To evaluate how execution time scales with increasing workload, we compare the total execution time of the sequential and hybrid implementations for a representative configuration using 8 worker processes, a single input split, and automatic chunk sizing. Figure 3 reports the resulting trends.

The results show that execution time grows approximately linearly with the size of the input dataset for both implementations. However, the hybrid approach exhibits a significantly lower growth rate, leading to an increasing performance gap as dataset size grows. In the tested configuration, total execution time increases from about 8.93 s to 102.54 s for the sequential implementation when moving from 2M to 20M elements, whereas the hybrid implementation grows only from about 3.20 s to 24.31 s over the same range.

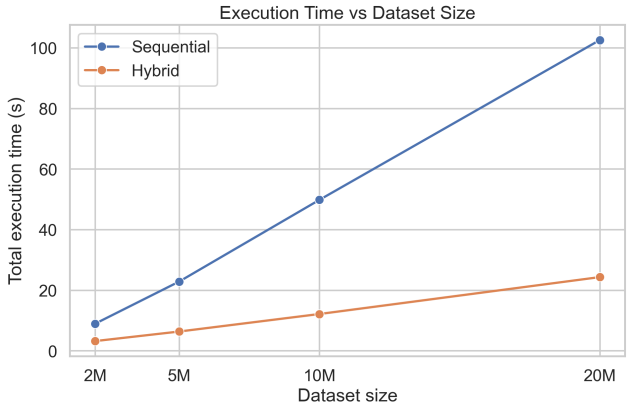

Figure 3. Execution time as a function of dataset size for the sequential and hybrid implementations. Results refer to a configuration with 8 worker processes, a single input split, and automatic chunk sizing for the hybrid version.

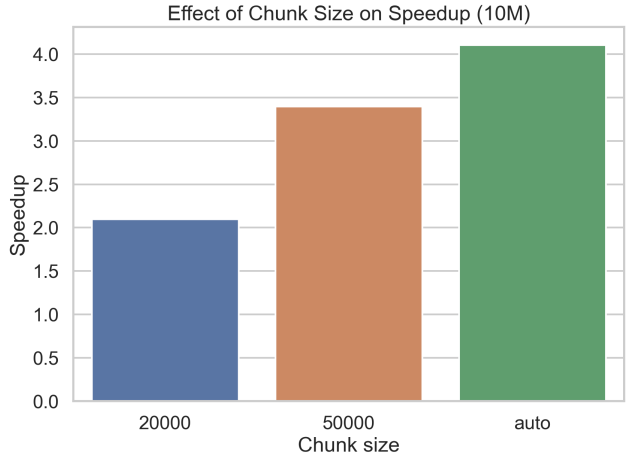

Figure 4. Effect of chunk size on speedup for a dataset of 10 million elements, using 8 worker processes and a single input split.

These results confirm that the proposed hybrid approach is particularly effective for large-scale workloads, where the benefits of parallelization increasingly outweigh the fixed overheads of multiprocessing and task coordination.

Effect of Chunk Size. Chunk size plays a crucial role in determining the overall efficiency of the parallel execution. Figure 4 illustrates the impact of different chunk size configurations on end-to-end performance, measured in terms of speedup, for a workload of 10 million elements using 8 worker processes and a single input split.

The results clearly show that a small fixed chunk size of 20,000 elements leads to poor

performance, with speedup around $2.10\times$. Increasing the chunk size to 50,000 improves performance substantially, yielding approximately $3.40\times$ speedup. The automatic chunking strategy performs best, reaching about $4.11\times$.

This behavior highlights the importance of balancing computational granularity and scheduling overhead. When chunks are too small, the cost of task creation, inter-process communication, and synchronization becomes too high relative to the useful work performed by each worker. Automatic chunking provides a robust and adaptive solution that simplifies configuration while delivering the best overall results in the tested setting.

5.6. Comparison of Parallelization Strategies and Bottleneck Analysis

This section compares the two execution strategies considered in this work and discusses the main factors that limit scalability. Since the project focuses on a single parallel design, the comparison is performed between the sequential baseline and the proposed hybrid parallel implementation.

Sequential vs Hybrid Parallel Strategy. The sequential implementation serves as a baseline reference and executes all phases of the Bloom Filter workflow—data loading, hash computation, filter construction, and membership queries—within a single process. This approach benefits from minimal orchestration overhead and a simple control flow, but it does not scale to large datasets due to the CPU-bound nature of repeated hash computations.

The hybrid implementation combines asynchronous I/O management and multiprocessing to address these limitations. Experimental results show that this design significantly outperforms the sequential baseline for sufficiently large datasets. In particular, the best end-to-end results are obtained with automatic chunk sizing and 8 worker processes, while the compute-only analysis shows that the underlying computational kernel scales even better than the application-level timing suggests.

Bottleneck Analysis and Scalability Limits. Despite the observed performance improvements, the hybrid implementation does not scale indefinitely. The results reveal a clear saturation point around 8 worker processes. Increasing the number of workers beyond this value does not improve performance and often causes a slight regression, both in end-to-end and compute-only measurements.

This behavior is primarily due to the overhead associated with multiprocessing. As the number of worker processes increases, costs related to task scheduling, process management, context switching, memory contention, and inter-process communication grow significantly. These overheads eventually limit the gains obtained from additional parallelism.

Task granularity is another major factor. The chunk-size experiments show that excessively small chunks can strongly degrade performance. This effect becomes even more evident for larger datasets: for example, on the 20M dataset with a single input split, chunk size 20,000 leads to speedup below 1 in several configurations, meaning that the parallel version becomes slower than the sequential baseline. This confirms that poor granularity can completely offset the benefits of multiprocessing.

Finally, the difference between end-to-end and compute-only speedup highlights the impact of non-computational costs. Since the compute-only curves are consistently above the end-to-end ones, part of the observed scalability limit must be attributed not to the Bloom Filter computation itself, but to file I/O, parsing, and coordination overheads.

Overall, the bottleneck analysis indicates that the scalability of the proposed solution is constrained not by the Bloom Filter algorithm itself, but by the overhead of multiprocessing and system-level resource management. These results highlight the importance of balanced parallelism, careful chunk sizing, and an experimen-

tal methodology able to distinguish between full-application behavior and the computational core.

6. Discussion

6.1. Interpretation of Results and Theoretical Comparison

The updated experimental results confirm that the proposed hybrid parallelization strategy provides substantial performance improvements over the sequential baseline, especially for medium and large datasets. From the end-to-end perspective, performance improves consistently as the number of worker processes increases from 1 to 8, after which a saturation effect becomes visible. The best end-to-end configuration is obtained with 8 worker processes and automatic chunk sizing, reaching speedups of about $4.11\times$ on the 10M dataset and $4.22\times$ on the 20M dataset.

The compute-only analysis strengthens this interpretation. When file reading and parsing overhead are excluded, the same configurations achieve even higher speedups, reaching approximately $5.59\times$ for 10M and $5.75\times$ for 20M. This shows that the computational core of the hybrid Bloom Filter scales more effectively than the full application workflow, and that part of the gap between theoretical and observed application-level speedup is due to non-computational overheads.

From a theoretical perspective, the observed behavior is consistent with Amdahl's Law. While the Bloom Filter algorithm is highly parallelizable at the level of input elements, some portions of the hybrid workflow remain effectively sequential or introduce unavoidable overhead, including task coordination, inter-process communication, and result aggregation. These components limit the maximum achievable end-to-end speedup and explain why the gains saturate beyond 8 worker processes.

At the same time, the results also align well with Gustafson's Law. As the problem size increases from 2M to 20M elements, both end-to-end and compute-only speedup improve significantly. This indicates that larger workloads better amortize the fixed costs of task orchestration and multiprocessing, allowing the parallel fraction of the computation to dominate more clearly.

Overall, the experimental evidence suggests that the adopted hybrid strategy is well suited for large-scale Bloom Filter operations in Python. The end-to-end analysis demonstrates practical performance gains at the application level, while the compute-only analysis reveals that the underlying computational kernel scales even more effectively than the global timing alone would suggest.

6.2. Overhead Analysis, Limitations, and Lessons Learned

Despite the achieved performance gains, the results clearly highlight the impact of parallelization overhead. The comparison between end-to-end and compute-only measurements shows that the hybrid implementation is affected not only by the cost of hash computation, but also by file handling, task scheduling, inter-process communication, synchronization, and aggregation. This explains why the compute-only speedup curves are systematically above the corresponding end-to-end curves.

A first important lesson is that multiprocessing overhead is not negligible and must be balanced carefully against workload size. For small datasets, such as 2M elements, the hybrid approach still improves performance, but the gains remain limited compared to larger workloads. In contrast, for 10M and 20M datasets, the amount of useful computation is large enough to amortize most of the coordination overhead, making the hybrid solution significantly more effective.

A second key lesson concerns task granularity. Chunk size has a strong impact on performance: when chunks are too small, the cost of scheduling and communication grows excessively and can severely reduce speedup. This effect is particularly evident in the configurations using chunk size 20,000, which perform substantially worse than both chunk size 50,000 and the automatic chunking strategy. In some 20M configurations, overly fine-grained chunking even makes the parallel implementation slower than the se-

quential baseline, confirming that poor granularity can completely offset the benefits of parallelism.

Another important limitation arises from hardware and runtime constraints. On the tested machine, performance saturates around 8 worker processes, even though 16 hardware threads are available. Beyond this point, the additional processes introduce more contention than useful throughput. This suggests that, for this type of workload and platform, the optimal number of processes is closer to the number of physical cores than to the number of hardware threads.

A further lesson learned is methodological. Measuring only the global execution time provides an application-level view, but it does not fully explain where the scalability limits originate. The introduction of compute-only timing proved valuable because it made it possible to distinguish the scalability of the algorithmic core from the overheads of I/O and orchestration. In this sense, the two timing perspectives are not alternatives but complementary tools for understanding performance.

Overall, these results show that effective parallelization in Python is not simply a matter of increasing the number of worker processes. Good performance depends on balancing workload size, task granularity, and available hardware resources, while also accounting for the runtime overhead introduced by multiprocessing and asynchronous coordination.

7. Conclusions

This work presented and evaluated a hybrid Bloom Filter implementation based on asynchronous I/O and multiprocessing in Python. The proposed approach successfully overcomes the limitations imposed by the Global Interpreter Lock for CPU-bound tasks and enables effective parallel execution on multi-core systems.

The experimental evaluation shows that the hybrid implementation achieves stable and repeatable end-to-end speedups slightly above $4\times$ for large datasets (10M and 20M elements), significantly reducing execution time compared to the sequential baseline. The best overall configuration consistently uses 8 worker processes combined with automatic chunk sizing, which balances computational efficiency and overhead. The additional compute-only analysis further shows that the computational core scales even better, exceeding $5.5\times$ speedup on the largest workloads.

These results confirm that the proposed hybrid strategy is an effective and scalable solution for large-scale Bloom Filter construction and querying in Python, while also highlighting the importance of distinguishing between full-application timing and pure computational scalability.

References

[1] B. H. Bloom, *Space/Time Trade-offs in Hash Coding with Allowable Errors*, Communications of the ACM, vol. 13, no. 7, pp. 422–426, 1970.

[2] A. Z. Broder and M. Mitzenmacher, *Network Applications of Bloom Filters: A Survey*, Internet Mathematics, vol. 1, no. 4, pp. 485–509, 2004.

[3] G. M. Amdahl, *Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities*, AFIPS Conference Proceedings, vol. 30, pp. 483–485, 1967.

[4] J. L. Gustafson, *Reevaluating Amdahl's Law*, Communications of the ACM, vol. 31, no. 5, pp. 532–533, 1988.

[5] Python Software Foundation, *multiprocessing — Process-based parallelism*, Python 3 Documentation. Available: `https://docs.python.org/3/library/multiprocessing.html`